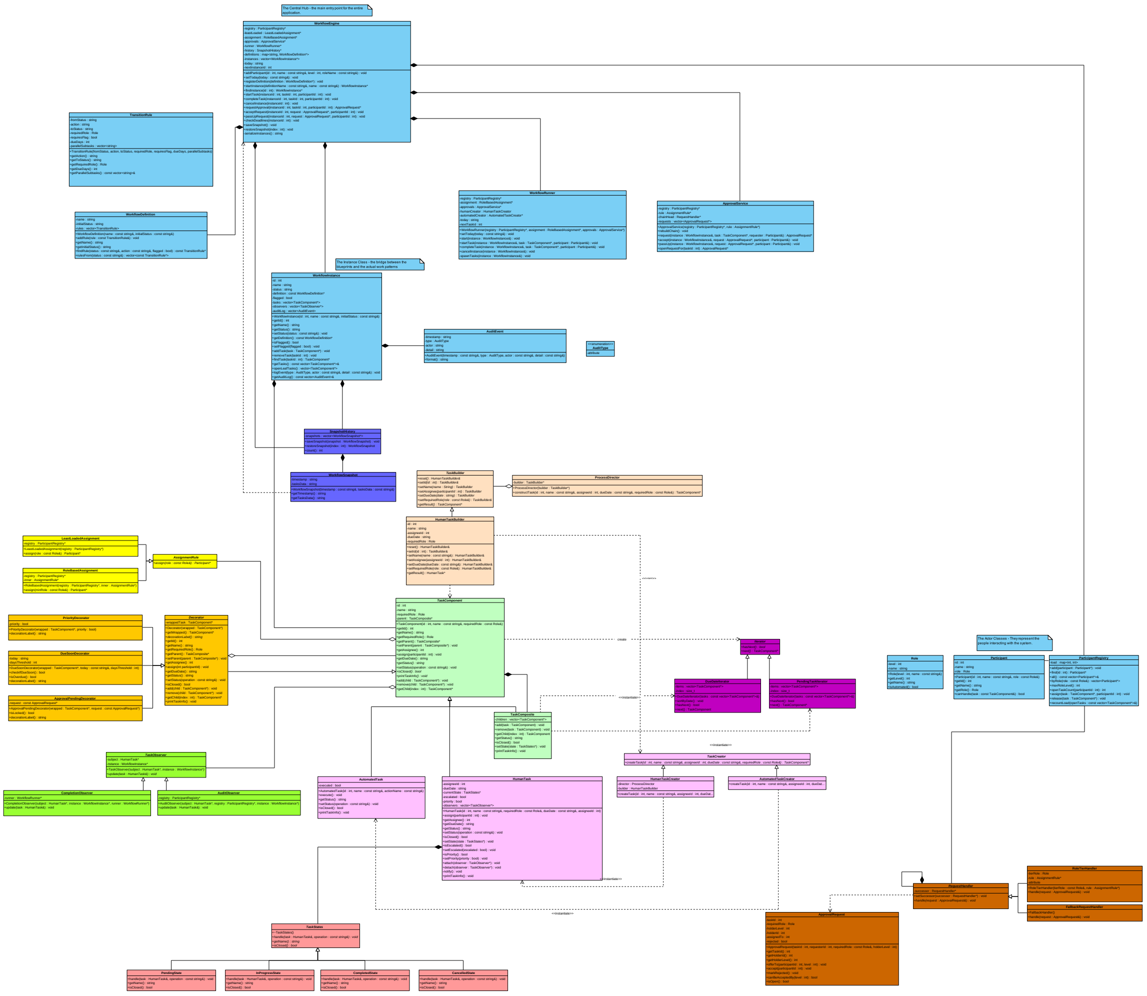
Visual Paradigm Standard(Leandro George(University of Pretoria))


The Central Hub - the main entry point for the entire application.
WorkflowEngine
-registry : ParticipantRegistry*
-leastLoaded : LeastLoadedAssignment*
-assignment : RoleBasedAssignment*
-approvals : ApprovalService*
-runner : WorkflowRunner*
-history : SnapshotHistory*
-definitions : map<string, WorkflowDefinition*>
-instances : vector<WorkflowInstance*>
-today : string
-nextInstanceId : int
+addParticipant(id : int, name : const string&, level : int, roleName : const string&) : void
+setToday(today : const string&) : void
+registerDefinition(definition : WorkflowDefinition*) : void
+startInstance(definitionName : const string&, name : const string&) : WorkflowInstance*
+findInstance(id : int) : WorkflowInstance*
+startTask(instanceId : int, taskId : int, participantId : int) : void
+completeTask(instanceId : int, taskId : int, participantId : int) : void
+cancelInstance(instanceId : int) : void
+requestApproval(instanceId : int, taskId : int, participantId : int) : ApprovalRequest*
+acceptRequest(instanceId : int, request : ApprovalRequest*, participantId : int) : void
+passUpRequest(instanceId : int, request : ApprovalRequest*, participantId : int) : void
+checkDeadlines(instanceId : int) : void
+saveSnapshot() : void
+restoreSnapshot(index : int) : void
-serializeInstances() : string
TransitionRule
-fromStatus : string
-action : string
-toStatus : string
-requiredRole : Role
-requiresFlag : bool
-dueDays : int
-parallelSubtasks : vector<string>
+TransitionRule(fromStatus, action, toStatus, requiredRole, requiresFlag, dueDays, parallelSubtasks)
+getAction() : string
+getToStatus() : string
+getRequiredRole() : Role
+getDueDays() : int
+getParallelSubtasks() : const vector<string>&
WorkflowDefinition
-name : string
-initialStatus : string
-rules : vector<TransitionRule>
+WorkflowDefinition(name : const string&, initialStatus : const string&)
+addRule(rule : const TransitionRule&) : void
+getName() : string
+getInitialStatus() : string
+findRule(status : const string&, action : const string&, flagged : bool) : const TransitionRule*
+rulesFrom(status : const string&) : vector<const TransitionRule*>
WorkflowRunner
-registry : ParticipantRegistry*
-assignment : RoleBasedAssignment*
-approvals : ApprovalService*
-humanCreator : HumanTaskCreator
-automatedCreator : AutomatedTaskCreator*
-today : string
-nextTaskId : int
+WorkflowRunner(registry : ParticipantRegistry*, assignment : RoleBasedAssignment*, approvals : ApprovalService*)
+setToday(today : const string&) : void
+start(instance : WorkflowInstance&) : void
+startTask(instance : WorkflowInstance&, task : TaskComponent*, participant : Participant&) : void
+completeTask(instance : WorkflowInstance&, task : TaskComponent*, participant : Participant&) : void
+cancelInstance(instance : WorkflowInstance&) : void
-spawnTasks(instance : WorkflowInstance&) : void
ApprovalService
-registry : ParticipantRegistry*
-rule : AssignmentRule*
-chainHead : RequestHandler*
-requests : vector<ApprovalRequest*>
+ApprovalService(registry : ParticipantRegistry*, rule : AssignmentRule*)
+rebuildChain() : void
+request(instance : WorkflowInstance&, task : TaskComponent*, requester : Participant&) : ApprovalRequest*
+accept(instance : WorkflowInstance&, request : ApprovalRequest*, participant : Participant&) : void
+passUp(instance : WorkflowInstance&, request : ApprovalRequest*, participant : Participant&) : void
+openRequestFor(taskId : int) : ApprovalRequest*
The Instance Class - the bridge between the blueprints and the actual work patterns
WorkflowInstance
-id : int
-name : string
-status : string
-definition : const WorkflowDefinition*
-flagged : bool
-tasks : vector<TaskComponent*>
-observers : vector<TaskObserver*>
-auditLog : vector<AuditEvent>
+WorkflowInstance(id : int, name : const string&, initialStatus : const string&)
+getId() : int
+getName() : string
+getStatus() : string
+setStatus(status : const string&) : void
+getDefinition() : const WorkflowDefinition*
+isFlagged() : bool
+setFlagged(flagged : bool) : void
+addTask(task : TaskComponent*) : void
+removeTask(taskId : int) : void
+findTask(taskId : int) : TaskComponent*
+getTasks() : const vector<TaskComponent*>&
+openLeafTasks() : vector<TaskComponent*>
+logEvent(type : AuditType, actor : const string&, detail : const string&) : void
+getAuditLog() : const vector<AuditEvent>&
AuditEvent
-timestamp : string
-type : AuditType
-actor : string
-detail : string
+AuditEvent(timestamp : const string&, type : AuditType, actor : const string&, detail : const string&)
+format() : string
<<enumeration>>
AuditType
-attribute
SnapshotHistory
-snapshots : vector<WorkflowSnapshot*>
+saveSnapshot(snapshot : WorkflowSnapshot) : void
+restoreSnapshot(index : int) : WorkflowSnapshot
+count() : int
WorkflowSnapshot
-timestamp : string
-tasksData : string
+WorkflowSnapshot(timestamp : const string&, tasksData : const string&)
+getTimestamp() : string
+getTasksData() : string
TaskBuilder
+reset() : HumanTaskBuilder&
+setId(id : int) : TaskBuilder&
+setName(name : String) : TaskBuilder
+setAssignee(participantId : int) : TaskBuilder
+setDueDate(date : string) : TaskBuilder
+setRequiredRole(role : const Role&) : TaskBuilder&
+getResult() : TaskComponent*
ProcessDirector
-builder : TaskBuilder*
+ProcessDirector(builder : TaskBuilder*)
+constructTask(id : int, name : const string&, assigneeId : int, dueDate : const string&, requiredRole : const Role&) : TaskComponent*
HumanTaskBuilder
-id : int
-name : string
-assigneeId : int
-dueDate : string
-requiredRole : Role
+reset() : HumanTaskBuilder&
+setId(id : int) : TaskBuilder&
+setName(name : const string&) : HumanTaskBuilder&
+setAssignee(assigneeId : int) : HumanTaskBuilder&
+setDueDate(dueDate : const string&) : HumanTaskBuilder&
+setRequiredRole(role : const Role&) : HumanTaskBuilder&
+getResult() : HumanTask*
<<uses>>
LeastLoadedAssignment
-registry : ParticipantRegistry*
+LeastLoadedAssignment(registry : ParticipantRegistry*)
+assign(role : const Role&) : Participant*
RoleBasedAssignment
-registry : ParticipantRegistry*
-inner : AssignmentRule*
+RoleBasedAssignment(registry : ParticipantRegistry*, inner : AssignmentRule*)
+assign(minRole : const Role&) : Participant*
AssignmentRule
+assign(role : const Role&) : Participant*
PriorityDecorator
-priority : bool
+PriorityDecorator(wrapped : TaskComponent*, priority : bool)
+decorationLabel() : string
DueSoonDecorator
-today : string
-daysThreshold : int
+DueSoonDecorator(wrapped : TaskComponent*, today : const string&, daysThreshold : int)
+checkIfDueSoon() : bool
+isOverdue() : bool
+decorationLabel() : string
ApprovalPendingDecorator
-request : const ApprovalRequest*
+ApprovalPendingDecorator(wrapped : TaskComponent*, request : const ApprovalRequest*)
+isLocked() : bool
+decorationLabel() : string
Decorator
-wrappedTask : TaskComponent*
+Decorator(wrapped : TaskComponent*)
+getWrapped() : TaskComponent*
+decorationLabel() : string
+getId() : int
+getName() : string
+getRequiredRole() : Role
+getParent() : TaskComposite*
+setParent(parent : TaskComposite*) : void
+getAssignee() : int
+assign(int participantId) : void
+getDueDate() : string
+getStatus() : string
+setStatus(operation : const string&) : void
+isClosed() : bool
+add(child : TaskComponent*) : void
+remove(child : TaskComponent*) : void
+getChild(index : int) : TaskComponent*
+printTaskInfo() : void
TaskComponent
-id : int
-name : string
-requiredRole : Role
-parent : TaskComposite*
+TaskComponent(id : int, name : const string&, requiredRole : const Role&)
+getId() : int
+getName() : string
+getRequiredRole() : Role
+getParent() : TaskComposite*
+setParent(parent : TaskComposite*) : void
+getAssignee() : int
+assign(participantId : int) : void
+getDueDate() : string
+getStatus() : string
+setStatus(operation : const string&) : void
+isClosed() : bool
+printTaskInfo() : void
+add(child : TaskComponent*) : void
+remove(child : TaskComponent*) : void
+getChild(index : int) : TaskComponent*
create
Iterator
+hasNext() : bool
+next() : TaskComponent*
DueDateIterator
-items : vector<TaskComponent*>
-index : size_t
+DueDateIterator(tasks : const vector<TaskComponent*>&)
+sortByDate() : void
+hasNext() : bool
+next() : TaskComponent
PendingTaskIterator
-items : vector<TaskComponent*>
-index : size_t
+DueDateIterator(tasks : const vector<TaskComponent*>&)
+hasNext() : bool
+next() : TaskComponent*
<<instantiate>>
TaskComposite
-children : vector<TaskComponent*>
+add(task : TaskComponent) : void
+remove(task : TaskComponent) : void
+getChild(index : int) : TaskComponent
+getStatus() : string
+isClosed() : bool
+setState(state : TaskStates*) : void
+printTaskInfo() : void
<<instantiate>>
TaskCreator
+createTask(id : int, name : const string&, assigneeId : int, dueDate : const string&, requiredRole : const Role&) : TaskComponent*
HumanTaskCreator
-director : ProcessDirector
-builder : HumanTaskBuilder
+createTask(id : int, name : const string&, assigneeId : int, dueDat...
AutomatedTaskCreator
+createTask(id : int, name : const string&, assigneeId : int, dueDat...
TaskObserver
-subject : HumanTask*
-instance : WorkflowInstance*
+TaskObserver(subject : HumanTask*, instance : WorkflowInstance*)
+update(task : HumanTask&) : void
CompletionObserver
-runner : WorkflowRunner*
+CompletionObserver(subject : HumanTask*, instance : WorkflowInstance*, runner : WorkflowRunner*)
+update(task : HumanTask&) : void
AuditObserver
-registry : ParticipantRegistry*
+AuditObserver(subject : HumanTask*, registry : ParticipantRegistry*, instance : WorkflowInstance*)
+update(task : HumanTask&) : void
AutomatedTask
-executed : bool
+AutomatedTask(id : int, name : const string&, actionName : const string&)
+execute() : void
+getStatus() : string
+setStatus(operation : const string&) : void
+isClosed() : bool
+printTaskInfo() : void
HumanTask
-assigneeId : int
-dueDate : string
-currentState : TaskStates*
-escalated : bool
-priority : bool
-observers : vector<TaskObserver*>
+HumanTask(id : int, name : const string&, requiredRole : const Role&, dueDate : const string&, assigneeId : int)
+assign(participantId : int) : void
+getAssignee() : int
+getDueDate() : string
+getStatus() : string
+setStatus(operation : const string&) : void
+isClosed() : bool
+setState(state : TaskStates*) : void
+isEscalated() : bool
+setEscalated(escalated : bool) : void
+isPriority() : bool
+setPriority(priority : bool) : void
+attach(observer : TaskObserver*) : void
+detach(observer : TaskObserver*) : void
-notify() : void
+printTaskInfo() : void
<<instantiate>>
<<instantiate>>
TaskStates
+~TaskStates()
+handle(task : HumanTask&, operation : const string&) : void
+getName() : string
+isClosed() : bool
PendingState
+handle(task : HumanTask&, operation : const string&) : void
+getName() : string
+isClosed() : bool
InProgressState
+handle(task : HumanTask&, operation : const string&) : void
+getName() : string
+isClosed() : bool
CompletedState
+handle(task : HumanTask&, operation : const string&) : void
+getName() : string
+isClosed() : bool
CancelledState
+handle(task : HumanTask&, operation : const string&) : void
+getName() : string
+isClosed() : bool
The Actor Classes - They represent the people interacting with the system.
Role
-level : int
-name : string
+Role(level : int, name : const string&)
+getLevel() : int
+getName() : string
+isAutomated() : bool
Participant
-id : int
-name : string
-role : Role
+Participant(id : int, name : const string&, role : const Role&)
+getId() : int
+getName() : string
+getRole() : Role
+canHandle(task : const TaskComponent&) : bool
ParticipantRegistry
-load : map<int, int>
+add(participant : Participant*) : void
+find(id : int) : Participant*
+all() : const vector<Participant*>&
+byRole(role : const Role&) : vector<Participant*>
+maxRoleLevel() : int
+openTaskCount(participantId : int) : int
+assign(task : TaskComponent*, participantId : int) : void
+release(task : TaskComponent*) : void
+recountLoad(openTasks : const vector<TaskComponent*>&)
RequestHandler
-successor : RequestHandler*
+setSuccessor(successor : RequestHandler*) : void
+handle(request : ApprovalRequest&) : void
RoleTierHandler
-tierRole : Role
-rule : AssignmentRule*
-attribute
+RoleTierHandler(tierRole : const Role&, rule : AssignmentRule*)
+handle(request : ApprovalRequest&) : void
FallbackRequestHandler
+FallbackHandler()
+handle(request : ApprovalRequest&) : void
ApprovalRequest
-taskId : int
-requiredRole : Role
-holderLevel : int
-holderId : int
-assignedTo : int
-rejected : bool
+ApprovalRequest(taskId : int, requesterId : int, requiredRole : const Role&, holderLevel : int)
+getTaskId() : int
+getHolderId() : int
+getHolderLevel() : int
+offerTo(participantId : int, level : int) : void
+accept(participantId : int) : void
+markRejected() : void
+canBeAcceptedBy(level : int) : bool
+isOpen() : bool